

Making External API Calls with WebClient

`RestTemplate` is the traditional way to make HTTP calls in Spring. It is blocking. Every call holds the thread until the response arrives. In a WebFlux reactive pipeline, using `RestTemplate` blocks a Netty event loop thread, which is catastrophic. `WebClient` is the non-blocking, reactive replacement.



With virtual threads and Spring MVC you can continue using `RestTemplate` or the newer `RestClient` without any issues. The thread blocks but the VT unmounts so no OS thread is wasted. `WebClient` is mandatory only in a WebFlux reactive pipeline where you cannot block under any circumstance.

Setting Up WebClient as a Bean

Never create `WebClient` instances inline with `WebClient.create()` in your service classes. Always configure it as a Spring bean so timeouts, codecs, base URLs, and default headers are defined in one place.

```
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient orderServiceClient() {
        return WebClient.builder()
            .baseUrl("https://api.orderservice.com")
            .defaultHeader(HttpHeaders.CONTENT_TYPE,
                MediaType.APPLICATION_JSON_VALUE)
            .defaultHeader(HttpHeaders.ACCEPT,
                MediaType.APPLICATION_JSON_VALUE)
    }
}
```

```

        .build();
    }

    @Bean
    public WebClient notificationServiceClient() {
        return WebClient.builder()
            .baseUrl("https://api.notifications.com")
            .defaultHeader(HttpHeaders.CONTENT_TYPE,
                MediaType.APPLICATION_JSON_VALUE)
            .build();
    }
}

```

Basic GET Call

Continuing with our user and order example. The `UserService` needs to fetch orders from a separate `OrderService`.

```

@Service
@RequiredArgsConstructor
public class OrderServiceClient {

    private final WebClient orderServiceClient;

    public Flux<OrderResponse> getOrdersByUserId(Long userId)
    {
        return orderServiceClient.get()
            .uri("/api/v1/orders?userId={userId}", userId)
            .retrieve()
            .bodyToFlux(OrderResponse.class);
    }

    public Mono<OrderResponse> getOrderById(Long orderId) {
        return orderServiceClient.get()
    }
}

```

```

        .uri("/api/v1/orders/{id}", orderId)
        .retrieve()
        .bodyToMono(OrderResponse.class);
    }
}

```

- `.get()` sets the HTTP method
- `.uri()` supports path variables and query params with template syntax
- `.retrieve()` triggers the request and gives access to the response
- `.bodyToMono()` for single object, `.bodyToFlux()` for collection or stream



`RestClient`, introduced in Spring 6.1, gives you a similar fluent builder API to `WebClient` but is blocking. With virtual threads it is a simpler alternative for outbound calls if you do not need reactive operators like `retry`, `timeout`, `zip`, or streaming responses.

POST, PUT, DELETE Calls

```

public Mono<OrderResponse> createOrder(Long userId, CreateOrderRequest request) {
    return orderServiceClient.post()
        .uri("/api/v1/orders")
        .bodyValue(request)
        .retrieve()
        .bodyToMono(OrderResponse.class);
}

```

```

public Mono<OrderResponse> updateOrder(Long orderId, UpdateOrderRequest request) {
    return orderServiceClient.put()
        .uri("/api/v1/orders/{id}", orderId)
        .bodyValue(request)
        .retrieve()
}

```

```

        .bodyToMono(OrderResponse.class);
    }

    public Mono<Void> cancelOrder(Long orderId) {
        return orderServiceClient.delete()
            .uri("/api/v1/orders/{id}", orderId)
            .retrieve()
            .bodyToMono(Void.class);
    }

```

Handling HTTP Error Responses

By default `WebClient` throws `WebClientResponseException` for 4xx and 5xx responses. You should handle these explicitly using `onStatus` so you can map them to your own domain exceptions.

```

public Mono<OrderResponse> getOrderById(Long orderId) {
    return orderServiceClient.get()
        .uri("/api/v1/orders/{id}", orderId)
        .retrieve()
        .onStatus(HttpStatusCode::is4xxClientError, response ->
            response.bodyToMono(ErrorResponse.class)
                .flatMap(error -> Mono.error(
                    new OrderNotFoundException("Order
not found: " + orderId)))
        )
        .onStatus(HttpStatusCode::is5xxServerError, response ->
            Mono.error(new ServiceUnavailableException("O
rder service is down"))
        )
        .bodyToMono(OrderResponse.class);
}

```

- `onStatus` takes a predicate on the status code and a function that returns `Mono<Throwable>`
- You can read the error response body before deciding which exception to throw
- Chain multiple `onStatus` calls for fine-grained status handling

Timeout

Always set a timeout on every WebClient call. Without it, a slow downstream service holds your reactive pipeline open indefinitely.

```
public Mono<OrderResponse> getOrderById(Long orderId) {
    return orderServiceClient.get()
        .uri("/api/v1/orders/{id}", orderId)
        .retrieve()
        .bodyToMono(OrderResponse.class)
        .timeout(Duration.ofSeconds(3));
}
```

When the timeout fires, `TimeoutException` is thrown into the pipeline. Handle it:

```
.timeout(Duration.ofSeconds(3))
.onErrorMap(TimeoutException.class,
    ex -> new ServiceUnavailableException("Order service timed out"));
```

Retry

Transient failures like network blips or brief service restarts should be retried automatically. Use `retryWhen` with exponential backoff.

```
public Mono<OrderResponse> getOrderById(Long orderId) {
    return orderServiceClient.get()
        .uri("/api/v1/orders/{id}", orderId)
        .retrieve()
        .onStatus(HttpStatus::is5xxServerError, respo
```

```
nse ->
        Mono.error(new ServiceUnavailableException("Order service down"))
        .bodyToMono(OrderResponse.class)
        .timeout(Duration.ofSeconds(3))
        .retryWhen(Retry.backoff(3, Duration.ofMillis(500)))
        .filter(ex -> ex instanceof ServiceUnavailableException)
        .onRetryExhaustedThrow((spec, signal) ->
            new ServiceUnavailableException("Order service down after retries")));
}
```

- `Retry.backoff(3, Duration.ofMillis(500))` retries up to 3 times with 500ms initial delay, doubling each time: 500ms, 1s, 2s
- `.filter()` retries only for specific exception types, not for 404s or validation errors
- `.onRetryExhaustedThrow()` controls what exception is thrown when all retries are exhausted



With virtual threads and `RestClient` you get retry support through Resilience4j `@Retry` annotation, which is arguably simpler to configure.

Fallback

When a downstream service is down and retries are exhausted, provide a fallback instead of propagating the error to the client.

```
public Flux<OrderResponse> getOrdersByUserId(Long userId) {
    return orderServiceClient.get()
        .uri("/api/v1/orders?userId={userId}", userId)
        .retrieve()
        .bodyToFlux(OrderResponse.class)
}
```

```

        .timeout(Duration.ofSeconds(3))
        .retryWhen(Retry.backoff(2, Duration.ofMillis(300)))

        .onErrorResume(ex -> {
            log.warn("Order service unavailable for user {}", returning empty", userId);
            return Flux.empty();
        });
    }
}

```

- `onErrorResume` switches to a fallback publisher when an error occurs
- `onErrorReturn` returns a single fallback value instead of a publisher
- `defaultIfEmpty` handles the case where the upstream returns empty but no error

```

// return cached or default data as fallback
.onErrorResume(ServiceUnavailableException.class,
    ex -> cacheService.getCachedOrders(userId))

```



Treat every external HTTP call as something that will eventually fail. Build timeout, retry, and fallback into every `WebClient` call from day one. Distributed systems fail in partial and unpredictable ways. A missing timeout or fallback on one downstream call can cascade and bring down your entire service under load.

Streaming Responses from External APIs

If the downstream service sends an SSE stream or a chunked JSON stream, `WebClient` handles it natively with `bodyToFlux`.

```

public Flux<PriceUpdate> streamPriceUpdates(String symbol) {
    return orderServiceClient.get()
        .uri("/api/v1/prices/stream?symbol={symbol}", symbol)
        .accept(MediaType.TEXT_EVENT_STREAM)

```

```
        .retrieve()  
        .bodyToFlux(PriceUpdate.class)  
        .doOnNext(update -> log.debug("Price update: {}",  
update))  
        .doOnError(ex -> log.error("Stream error: {}", e  
x.getMessage()));  
    }
```

This keeps the connection open and emits each event as it arrives from the upstream service.